

Report

1. 方法 (Method)

1.1 整体模型结构

本项目以课程提供的 decoder-only Transformer baseline 为基础，在保持 causal language modeling 框架不变的前提下，对模型深度、FFN 容量、位置编码、归一化方式以及训练策略进行了修改。

最终模型的输入首先经过 token embedding，将离散 token ID 映射为连续的 hidden representation。与 baseline 不同，最终模型不再使用 learned absolute positional embedding，而是在 Attention 内部使用 Rotary Position Embedding (RoPE) 编码位置信息。

随后，token representation 依次经过 3 个 Transformer Block。每个 Block 均采用 pre-normalization residual structure，即先进行归一化，再经过 Attention 或 FFN，最后将输出加回 residual stream。

一个 Block 的基本计算过程可以表示为：

$$x_{\text{attn}} = x + \text{Dropout}(\text{Attention}(\text{RMSNorm}(x)))$$

然后：

$$x_{\text{out}} = x_{\text{attn}} + \text{Dropout}(\text{FFN}(\text{RMSNorm}(x_{\text{attn}})))$$

所有 Block 完成后，再经过一个最终 RMSNorm，并通过 language-modeling head 映射到词表 logits。

最终模型的 token embedding 与 output head 共享权重，以减少独立参数数量。当前实现中，模型主体宽度为 160，使用 4 个 attention heads。

1.2 Attention: RoPE 与 QK-Norm

每个 Transformer Block 首先对输入进行 RMSNorm，然后通过一个线性层同时生成 Query、Key 和 Value：

$$Q, K, V = W_{\text{qkv}} \cdot \text{RMSNorm}(x)$$

得到 Q、K、V 后，将其划分为多个 attention heads。

与 baseline 使用 learned absolute positional embedding 不同，本模型将 RoPE 直接应用到 Query 和 Key：

$$Q_r = \text{RoPE}(Q)$$

$$K_r = \text{RoPE}(K)$$

RoPE 通过根据 token position 对每个 attention head 中成对的 feature dimension 进行旋转，使位置信息直接参与 Query-Key similarity 的计算。当前实现对 Q 和 K 进行 RoPE 变换，而 V 保持不变。

在 RoPE 之后，对 Query 和 Key 分别使用 QK-Norm：

$$Q_{\text{hat}} = \text{Norm}(Q_r)$$

$$K_{\text{hat}} = \text{Norm}(K_r)$$

然后使用 causal scaled dot-product attention：

$$\text{Attention}(Q, K, V) = \text{softmax}(Q_{\text{hat}} \cdot K_{\text{hat}}^T / \sqrt{d_{\text{head}}} + \text{causal_mask}) \cdot V$$

其中 causal mask 用于保证当前位置不能访问未来 token。

Attention 的多头输出重新拼接后，通过 output projection 映射回 model width，并以 residual connection 的方式加回输入。

因此，本项目对 Attention 的主要优化不是单纯增加 attention dimension，而是重点改进 positional representation 以及 Query-Key interaction 的数值稳定性。最终有效 Attention dimension 与 model width 保持一致，即 160。

1.3 非对称 FFN 设计

Baseline 中不同 Transformer Block 使用相同的 FFN expansion ratio。本项目根据各层在实验中的不同表现，对 FFN capacity 进行了非对称分配。

第一个 Block 使用较大的 SwiGLU FFN：

$$160 \rightarrow 5 \times 160 \rightarrow 160$$

SwiGLU 的计算过程可以表示为：

$$\text{gate} = \text{SiLU}(W_g \cdot x)$$

$$\text{value} = W_v \cdot x$$

$$\text{output} = W_o \cdot (\text{gate} \odot \text{value})$$

其中 $\odot$ 表示 element-wise multiplication。

与传统 GELU FFN 相比，SwiGLU 使用额外的 gating branch 控制中间 feature 的传播。

Block 1 仍使用 SwiGLU，但将 hidden dimension 缩小为：

$$160 \rightarrow 2 \times 160 \rightarrow 160$$

因此，更多 FFN capacity 被分配给前面的 Block，而后层使用更加紧凑的 FFN。

额外加入的 Block 2 同样使用较小的 2d hidden dimension，但保留普通 GELU FFN：

$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 \cdot x)$$

其维度同样为：

$$160 \rightarrow 320 \rightarrow 160$$

因此，最终模型没有强制所有 Block 使用完全相同的 FFN，而是采用了不同的 FFN capacity 和 activation design。

1.4 RMSNorm

模型中的主要归一化方法由 baseline 的 LayerNorm 修改为 RMSNorm。

对于一个 d 维输入向量 x ，RMSNorm 首先计算：

$$\text{RMS}(x) = \sqrt{\text{mean}(x^2) + \epsilon}$$

然后进行归一化：

$$\text{RMSNorm}(x) = g \odot x / \text{RMS}(x)$$

其中 g 是可学习的 scaling parameter。

与 LayerNorm 不同，RMSNorm 不执行 mean centering，而主要控制 activation 的整体数值尺度。

当前实现还在 FP16 或 BF16 输入情况下，将 RMS 的计算临时转换到 FP32，以提高数值稳定性。

1.5 Residual Dropout

最终模型在 Attention branch 和 FFN branch 的输出加回 residual stream 之前加入 Dropout。

计算方式为：

$$x = x + \text{Dropout}(\text{attention_output})$$

以及：

$$x = x + \text{Dropout}(\text{ffn_output})$$

最终配置的 dropout probability 为：

$$p = 0.05$$

即训练过程中随机屏蔽约 5% 的 residual branch activation。

Dropout 只在训练阶段启用；在 validation 和 inference 阶段关闭，因此不会在推理阶段随机删除 feature。

最终实验中，加入 Dropout 后模型的泛化性能进一步改善。最终配置文件中的 dropout 设置为 0.05。

1.6 输出层与参数共享

所有 Transformer Block 完成后，hidden representation 首先经过最终 RMSNorm。

之后使用一个不带 bias 的 linear head，将 hidden representation 从 model width 映射到 vocabulary dimension：

```
160 → 2048
```

得到每个位置对 2048 个 vocabulary token 的 logits。

同时，模型令：

```
LM_head.weight = Token_embedding.weight
```

即输入 embedding 和输出 language-modeling head 共享同一份权重。

这种 weight tying 可以减少模型的独立参数数量，同时使 token 的输入表示和输出预测使用同一个 representation space。

1.7 Training Strategy

最终模型采用 AdamW optimizer。

最终使用的主要训练参数为：

- Optimizer: AdamW
- Training steps: 9600
- Batch size: 32
- Context length: 256
- Peak learning rate: 0.0013
- Weight decay: 0.05
- Gradient clipping: 1.0
- Dropout: 0.05

训练使用 next-token prediction cross-entropy loss：

```
Loss = CrossEntropy(model(tokens[0:T]), tokens[1:T+1])
```

也就是说，每个位置的模型输出用于预测下一个 token。

学习率采用 100-step linear warmup 加 cosine decay。

Warmup 部分为：

```
warmup_factor = min(1, step / 100)
```

Cosine 部分为：

```
cosine_factor = 0.2 + 0.8 × 0.5 × (1 + cos( $\pi \times \text{step} / \text{total\_steps}$ ))
```

最终学习率：

$$\text{lr} = 0.0013 \times \text{warmup_factor} \times \text{cosine_factor}$$

因此：

- 前 100 step，learning rate 从接近 0 线性增加；
- warmup 完成后，learning rate 使用 cosine schedule 逐渐降低；
- 最终学习率不会下降到 0，而是保留 peak learning rate 的约 20%。

AdamW 的 weight decay 最终设置为：

$$\text{weight_decay} = 0.05$$

同时使用 global gradient clipping：

$$\text{gradient_norm} \leq 1.0$$

避免异常大的 gradient update。

最终训练使用：

$$9600 \times 32 \times 256 = 78,643,200$$

个 training targets。最终实验采用 BF16 precision，模型参数量为 1,281,440。

2. 实验记录（Comparisons & Ablation）

	0	1	2	3	4	5	6	7	8
change (key_change)	Baseline (step=1200, batch=32)	去掉后两层 block	将 block1 的 ffn 部分变成 d-2d-d	对 block1 部分 ffn 部分从 ReLU 换成 SwiGLU	对 attention 部分的位置 表达做 RoPE	对 attention 部分做 QK-Norm	增大 token 的 width，从 128 变为 160	将 Block0 的 ffn 部分也从 ReLU 换成 SwiGLU	将 LayerNorm 换成 RMS-Norm
vali	2.0711	2.0809	2.0838	2.0736	1.9147	1.9096	1.8616	1.838	1.838
注：	加粗的部分为 key_change								

	9	10	11	12	13	14	15	16	17
--	---	----	----	----	----	----	----	----	----

change (key_change)	将Block0的ffn部分从d-4d-d变成d-5d-d	将training_step改为2400，batch_size改为16（总训练token不变）	加Block2（结构同Block1）	将decay从0.1改成0.05（step=2400 batch=16）	step=2400 batch=32（训练token数量*2）	step=4800 batch=16（训练token数量*4）	step=9600 batch=16（训练token数量*8）	加入dropout=0.05	step=9600 batch=32（训练token数量*8）
vali	1.824	1.784	1.760	1.7506	1.717	1.7017	1.6858	1.667	1.6458
注：	这里的12和17可作为对照（12的总训练token数与baseline一致）表明增加训练的token可以提升模型效果								

3. 分析（Analysis）

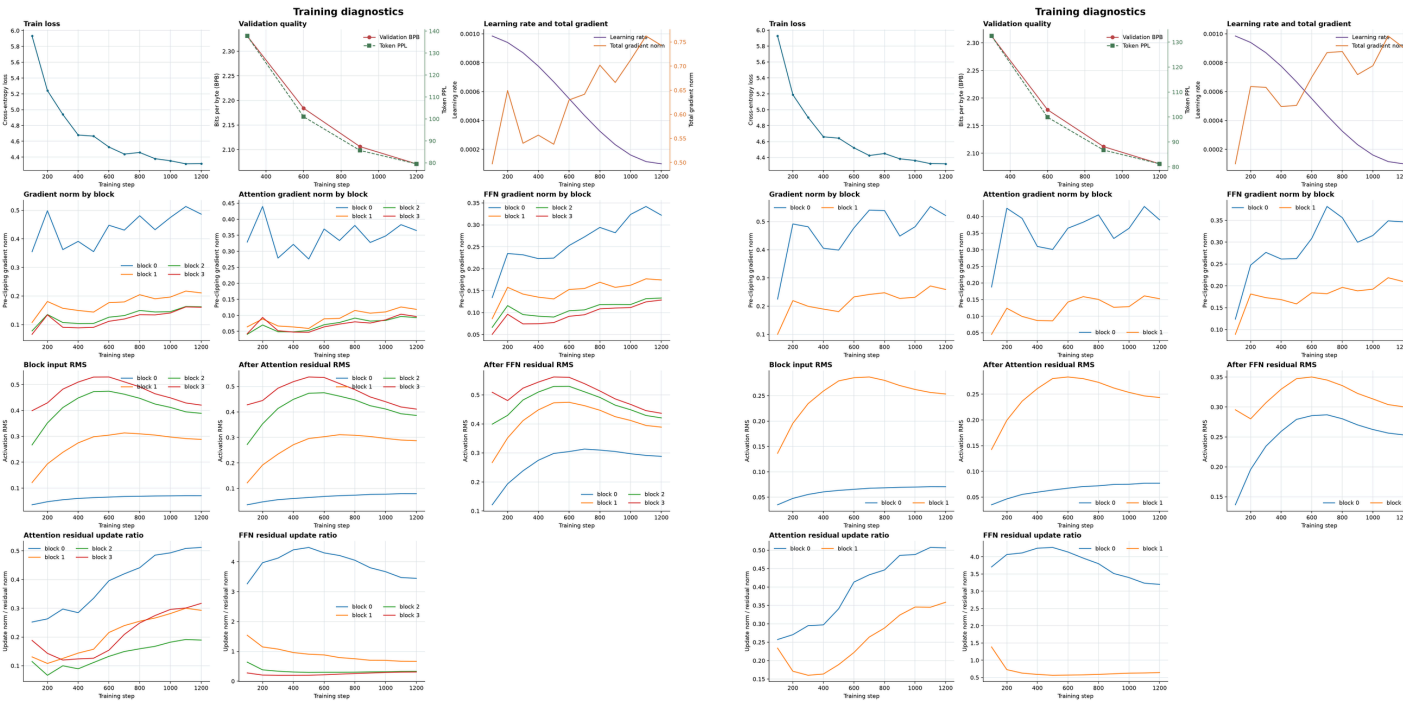
3.1 基于诊断信息的模型简化

优化过程从课程提供的 Transformer baseline 出发。相比直接增加模型规模，我首先分析了不同 Transformer Block 在训练过程中的行为，包括每个 Block 的梯度范数和残差更新比例。

残差更新比例定义为：

$$R=||\delta x||/||x||$$

其中，x表示进入某个子模块前的 residual stream，delta x表示 Attention 或 FFN 分支对其产生的更新。



在最初的四层模型中，后两层 Block 的梯度范数和 residual update ratio 都明显较小。这说明在原始结构下，后层对表示的边际修改相对较弱。因此，我首先移除了最后两个 Block，将网络深度从 4 层减少到 2 层。

该修改只造成了较小的 BPB 性能下降，但显著减少了参数量和计算开销。因此，后续实验以这个更加紧凑的两层结构作为基础，并将节省出的模型容量重新分配给更有价值的部分。

需要强调的是，这并不意味着后两层完全“无用”。实验只能说明：在当时的原始模型中，它们相对于所占用的计算资源，边际收益较低。

3.2 FFN 容量的重新分配

进一步分析发现，剩余两个 Block 中的 FFN 行为存在明显差异。Block 0 的 FFN residual update ratio 显著高于 Block 1，而 Block 1 FFN 的更新幅度较小。

因此，我没有尝试强制两个 FFN 具有相近的更新比例，而是将这种差异视为一种“不同层容量需求不同”的信号。

首先，我将 Block 1 的 FFN expansion ratio 从 (4d) 缩减到 (2d)。实验结果表明，BPB 不仅没有恶化，反而出现了小幅改善。这说明原来的 Block 1 FFN 存在一定的冗余。

随后，我将传统的 GELU FFN 改为 SwiGLU 结构。SwiGLU 通过额外的 gating mechanism，对中间特征进行选择调节，使 FFN 能够更加灵活地决定哪些 token-level feature 被保留和传递。

在后续实验中，Block 0 FFN 一直保持较高的 residual update ratio，因此进一步测试了增加其容量的效果。将 Block 0 FFN 从 (4d) 增加到 (5d) 后，BPB 进一步下降约 0.014–0.015。

这一结果说明，不同 Block 的 FFN 并不一定适合使用完全相同的 hidden expansion ratio。对当前模型而言，Block 0 更需要 FFN capacity，而 Block 1 的 FFN 可以保持更紧凑。

3.3 Attention 与位置表示的改进

之后，我对 Attention 部分进行了单独分析。

这一阶段最重要的改动是引入 Rotary Position Embedding (RoPE)。与单纯增加 Attention 参数量不同，RoPE 直接改变 query 与 key 中位置信息的表达方式，使 Attention 能够更加自然地编码 token 之间的相对位置关系。RoPE 是整个结构优化过程中效果最明显的单项修改之一，说明原始 positional representation 是当前模型的重要限制之一。

在此基础上，我进一步引入了 QK-Norm，对 query 和 key 在计算 attention score 之前进行归一化。与 RoPE 相比，QK-Norm 的收益较小，但仍带来了额外的 BPB 改善，因此被保留下来。

随后还测试了 Attention width 的扩大。全局增加 Attention capacity 只能带来有限收益，而只将 Block 1 的 attention width 从 160 增加到 288 时，validation BPB 反而恶化。

这一负结果说明：Residual update ratio 较大并不意味着该模块一定缺少 capacity。Block 1 Attention 虽然保持较高的 residual update ratio，说明它在 forward 中承担重要的信息整合功能，但增加其参数量并不能进一步提高性能。

3.4 模型 Width 与 Depth 调整

在删除冗余深度并重新分配 FFN 和 Attention 容量后，我进一步扩大了模型整体 width。

与只增加 Attention width 相比，提高整个 residual stream 的 dimension 带来了更明显的性能提升。这表明模型的主要容量限制并不仅存在于 Attention 内部，而是与整体 token representation 的表达能力有关。

模型 width 增大后，不仅 Attention 获得了更大的输入和输出表示空间，FFN、residual connection 以及中间 token feature 也同时获得了更大的 representation capacity。

然而，该改动也显著增加了计算时间，因为 Transformer 中的大量 Linear operation 的计算复杂度近似随 hidden width 的平方增长。因此，扩大 width 本质上是一种性能和计算开销之间的 trade-off。

随后，我还在已有结构基础上增加了第三个 Block，并使其结构与 Block 1 类似。该修改进一步降低了约 0.02 BPB，说明在前面的结构优化完成后，额外 depth 仍然能够提供一定收益。

但相比带来的额外计算时间，这一收益已经相对有限，表现出了明显的 diminishing returns。

3.5 Training Strategy 优化

在模型结构基本固定后，优化重点转向训练过程。

最重要的修改是重新调整 batch size 与 optimizer update 次数之间的关系。

原始训练设置为1200 个 optimization steps、batch size 为 32，随后改为2400*16。这样在总 processed training targets 基本不变的情况下，将 optimizer update 次数翻倍。

该修改使 BPB 下降约 0.06，是 training strategy 中最明显的单项提升之一。这说明原始训练设置下，模型并没有得到充分优化，更多、更细粒度的 parameter update 对当前模型更加有效。

之后继续测试2400*32，4800*16，9600*16，模型性能仍然能够继续改善，但是随着训练时间增加，收益逐渐减小，表现出明显的 diminishing returns。

除了训练长度之外，还测试了多种 optimizer 相关超参数：

- 提高 cosine decay 的 final learning-rate floor，收益非常小；
- 将 gradient clipping threshold 从 1.0 放宽到1.5后，BPB 略微恶化；
- 将 peak learning rate 从0.001 提高到约0.0013，仅带来约 0.007 BPB 的改善；
- 将 AdamW weight decay 从 0.1 降低到 0.05，仅带来约 0.004 BPB 改善，而提高到 0.2 则使 BPB 恶化。

这些实验表明，相比精细调整 optimizer hyperparameters，增加有效 optimization steps 对当前模型的影响更大。

3.6 Dropout Regularization

在模型进行了更充分的训练后，我进一步引入了 Dropout 作为 regularization mechanism。

原始模型中并没有 Dropout，因此该机制需要显式加入到 Transformer forward 中。其目的不是增加模型 capacity，而是在训练过程中随机屏蔽部分 feature activation，从而减少模型对特定 feature 或 computation path 的过度依赖。

在较长训练设置下，引入 Dropout 后，validation BPB 从1.6858下降到1.667，即改善约0.0188 BPB。这说明当模型训练更加充分后，regularization 开始变得有效。与早期阶段相比，此时模型已经不再主要受“训练不足”限制，而是开始能够从增强泛化能力的训练策略中获益。